

Dashboard Realtime — Cơ chế auto-update progress

MyMoneyWent · maingocanh1702

2026-05-13

Contents

Dashboard Realtime — Cơ chế auto-update progress	1
TL;DR	1
1. Kiến trúc 3 tầng	1
2. Tầng 1 — Server-side rebuild	2
3. Tầng 2 — Railway FastAPI serve	4
4. Tầng 3 — Browser-side live-poll JS	4
5. End-to-end flow (commit → browser update)	6
6. Security — GitHub PAT	6
7. Troubleshooting	7
8. Commit history (timeline)	8
9. Cải tiến tương lai	9
10. Tổng kết	9
Changelog	10

Dashboard Realtime — Cơ chế auto-update progress

Version: v1.0.0 **Ngày tạo:** 2026-05-13 **Live URL:** <https://mymoneywent-production.up.railway.app/dashboard>
Trạng thái: Active — shipped end-to-end 2026-05-13 **Mục đích:** Giải thích đầy đủ cách dashboard tự cập nhật tiến độ dự án trong realtime, không cần reload tay.

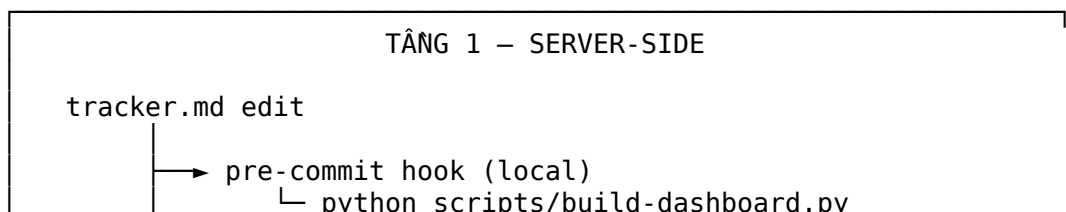
TL;DR

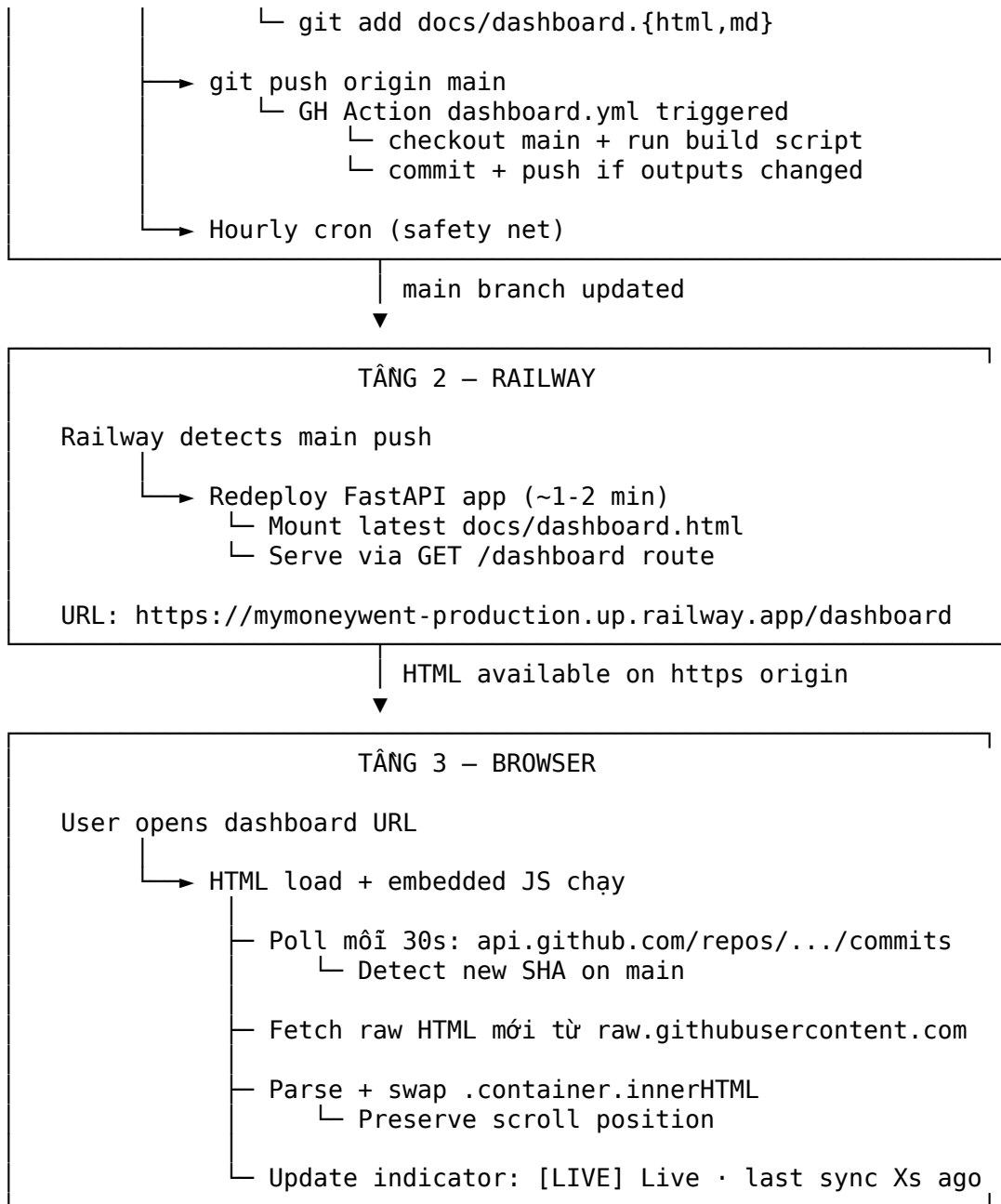
Dashboard `dashboard.html` cập nhật khi anh push code, qua **3 tầng phối hợp**:

1. **Server-side:** Mỗi push lên main → GitHub Action regenerate `dashboard.html` từ `implementation-tracker.md` + commit lại
2. **Railway hosts:** FastAPI route GET `/dashboard` serve file luôn mới sau redeploy
3. **Browser-side:** JS trong `dashboard.html` poll GitHub API mỗi 30s, phát hiện commit mới → fetch raw HTML → swap DOM (giữ scroll position)

Latency end-to-end: ~2-3 phút từ lúc anh git push đến lúc browser auto-refresh.

1. Kiến trúc 3 tầng





2. Tầng 1 — Server-side rebuild

2.1 Source of truth

Dashboard render từ một file duy nhất: docs/implementation-tracker.md. Đây là master status board mọi PR của dự án — mỗi row có (Phase, Wave, Feature, Status emoji, Branch, Gates, Notes). Script scripts/build-dashboard.py parse markdown này, enrich bằng git state (branch tồn tại không, commits ahead, fix() commits, push/pull state), rồi render thành dashboard.html + dashboard.md.

2.2 Cơ chế 1 — Pre-commit hook (local, anh trigger)

File .pre-commit-config.yaml có hook:

```
- id: build-dashboard
  name: Rebuild dashboard from tracker + git state
  entry: bash -c 'python scripts/build-dashboard.py && git add docs/dashboard.html docs/dashboard.md'
```

```
language: system
files: ^(docs/implementation-tracker\.md|scripts/build-dashboard\.py)$
pass_filenames: false
require_serial: true
```

Khi anh commit và staged files gồm tracker.md hoặc build-dashboard.py: 1. Hook chạy python scripts/build-dashboard.py 2. Script regen docs/dashboard.{html,md} 3. Hook auto-stage 2 files đó vào commit hiện tại 4. Commit complete với dashboard fresh nhúng cùng

Ưu điểm: zero latency — dashboard mới có ngay trong commit, không cần round-trip CI.

2.3 Cơ chế 2 — GitHub Action (sau push)

File .github/workflows/dashboard.yml:

```
on:
  push:
    branches: [main, 'feat/**', 'infra/**', 'chore/**', 'fix/**']
  schedule:
    - cron: '0 * * * *'
  workflow_dispatch:

concurrency:
  group: dashboard-rebuild
  cancel-in-progress: true

jobs:
  rebuild:
    if: ${github.event_name == 'schedule' || github.event_name == 'workflow_dispatch' || !con
    steps:
      - checkout main (ref: main, fetch-depth: 0)
      - run: python scripts/build-dashboard.py
      - if outputs changed:
          git commit -m "chore(dashboard): auto-rebuild — push from main (<sha>)"
          git push origin main
```

Trigger conditions: - Push lên main/feat/infra/chore/fix branches - Hourly cron 0 * * * * - Manual workflow_dispatch từ Actions tab

Anti-loop guard: skip nếu commit message chứa “auto-rebuild” (tránh bot tự trigger lại chính bot’s commits → infinite loop).

Concurrency: chỉ 1 run của dashboard-rebuild group chạy tại 1 thời điểm; runs cũ bị cancel khi run mới start.

2.4 Cơ chế 3 — Hourly cron (safety net)

Cron 0 * * * * rebuild mỗi giờ kể cả không có push. Catch các trường hợp: - Anh commit --no-verify (bypass pre-commit hook) - GH Action push trigger fail vì rate limit/network - Drift do edit tracker từ GitHub web UI

2.5 Tại sao 3 cơ chế?

Defense-in-depth. Single mechanism nào cũng có failure mode (anh quên hook, GH Action bị skip vì concurrency, cron miss). Cả 3 đảm bảo dashboard.html trên main luôn fresh trong tệp lăm 1 giờ.

3. Tầng 2 — Railway FastAPI serve

3.1 Tại sao cần host trên https?

Browser security: Chrome (và nhiều browser khác) **block** fetch() cross-origin từ file:// đến https://. Nếu anh mở dashboard.html local từ disk, JS polling sẽ fail với CORS error.

Solution: serve dashboard.html qua https origin → fetch tới api.github.com (cũng https) → same-origin protocol, no CORS issue.

3.2 FastAPI routes

main.py (commit b6b711b):

```
from pathlib import Path
from fastapi.responses import FileResponse, JSONResponse

ROOT = Path(__file__).resolve().parent
DASHBOARD_HTML = ROOT / "docs" / "dashboard.html"
DASHBOARD_MD = ROOT / "docs" / "dashboard.md"

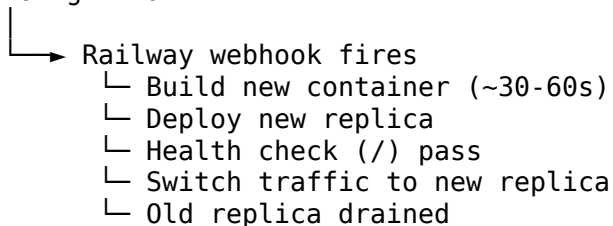
@app.get("/dashboard", include_in_schema=False)
async def serve_dashboard():
    if not DASHBOARD_HTML.exists():
        return JSONResponse({"error": "dashboard.html not found"}, status_code=404)
    return FileResponse(
        DASHBOARD_HTML,
        media_type="text/html",
        headers={"Cache-Control": "public, max-age=30"},
    )

@app.get("/dashboard.md", include_in_schema=False)
async def serve_dashboard_md():
    # Tương tự, serve markdown version
    ...
```

Key points: - FileResponse stream file từ disk — Railway có file mới nhất sau mỗi redeploy - Cache-Control: public, max-age=30 — browser/CDN cache 30s. Trade-off: realtime polling tự lo update, cache giảm Railway egress - include_in_schema=False — exclude khỏi /docs OpenAPI surface - 404 JSONResponse fallback nếu file missing

3.3 Railway redeploy lifecycle

git push origin main



Tổng ~1-2 phút. Sau đó /dashboard serve HTML mới nhất từ main.

4. Tầng 3 — Browser-side live-poll JS

4.1 Embed pattern

scripts/build-dashboard.py có một constant HTML_LIVE_JS — string JS được inject vào dashboard.html trước </body> mỗi lần build. Live-poll logic **không tách file riêng** — embed trực tiếp vào HTML để zero-

dependency, browser load 1 file là chạy được ngay.

Build script cũng có helper `_detect_repo_slug()` để parse git remote get-url origin → ra owner/repo slug, substitute vào JS template (thay placeholder `__REPO__`). Có nghĩa nếu anh fork repo, dashboard JS tự bind repo URL của fork đó, không cần edit code.

4.2 Poll loop

```
var REPO = 'maingocanh1702/MyMoneyWent';
var POLL_INTERVAL_MS = 30000;
var TRACK_PATH = 'docs/dashboard.html';

function fetchLatestSha() {
  var url = 'https://api.github.com/repos/' + REPO +
    '/commits?path=' + TRACK_PATH + '&per_page=1';
  return fetch(url, { headers: authHeaders() }).then(...).then(data => data[0].sha);
}

setInterval(check, POLL_INTERVAL_MS);
```

Mỗi 30s: 1. Gọi GitHub API GET /repos/{owner}/{repo}/commits?path=docs/dashboard.html&per_page=1
2. So sánh SHA mới với lastKnownSha 3. Nếu khác → có commit mới → fetch raw → swap DOM

4.3 DOM swap (preserve scroll)

```
function refreshDashboardDOM() {
  var url = 'https://raw.githubusercontent.com/' + REPO +
    '/main/docs/dashboard.html?t=' + Date.now();
  return fetch(url, { cache: 'no-store' }).then(r => r.text()).then(text => {
    var parser = new DOMParser();
    var newDoc = parser.parseFromString(text, 'text/html');
    var newContainer = newDoc.querySelector('.container');
    var oldContainer = document.querySelector('.container');
    if (newContainer && oldContainer) {
      var scrollY = window.scrollY;
      oldContainer.innerHTML = newContainer.innerHTML;
      window.scrollTo(0, scrollY);
    }
  });
}
```

Key behavior: - Fetch raw từ raw.githubusercontent.com (cache-bust với `?t=<timestamp>`) - Parse HTML qua DOMParser - Replace `.container.innerHTML` (preserve scroll Y position) - KHÔNG reload toàn trang — preserve script context + page state

4.4 Indicator state machine

State	Icon	Meaning	Trigger
init	[INIT]	Loading, first poll pending	Page load
live	[LIVE]	Polling OK, no new commits	Successful poll, SHA unchanged
syncing	[SYNC]	Detected new SHA, fetching raw	Mid-DOM-swap
error	[ERR]	API/network error	fetch() throw
rate-limit	[WARN]	60/hr unauth limit hit	API 403 + X-RateLimit-Remaining=0
paused	[PAUSE]	User halted polling	Esc key

Pulse animation live-pulse 2s ease-in-out infinite cho state live → visual feedback “đang sống”.

4.5 UX controls

- **Click indicator** → force-refresh ngay (skip 30s wait)
- **Esc key** → toggle pause/resume
- **localStorage github_pat** → set PAT cho 5000/hr rate (thay vì 60/hr unauth)

4.6 Edge cases handled

- **Rate limit**: return 'skipped' sentinel xuyên promise chain để KHÔNG overwrite [WARN] indicator với [LIVE] (Codex round 02 fix)
- **TRACK_PATH single, not array**: dashboard.html là downstream của tracker.md qua pre-commit hook → polling chỉ dashboard.html catch all updates, không cần double polling tracker (Codex round 01 fix)
- **Private repo + unauth**: 404 (GitHub policy không leak existence private repos) → indicator [ERR] cho đến khi anh set PAT

5. End-to-end flow (commit → browser update)

```
T+0s      anh: git push origin main
T+5s      └─> GitHub Action `dashboard.yml` trigger
T+10s      │   ├── checkout main
T+20s      │   ├── pip install (cached) + run build script
T+30s      │   └── if outputs changed: bot commit + push
T+30s      │       └─ "chore(dashboard): auto-rebuild - push from main (<sha>)"
T+30s      └─> Railway webhook fires (parallel to bot push)
T+60s      │   └─ Build container
T+90s      │   └─ Deploy + health check pass
T+90s      └─> Bot's commit hit main → trigger another Railway redeploy (anti-
loop in workflow đã skip rebuild)
T+150s      └─> browser tab dashboard polling
T+150s      │   ├── Fetch api.github.com/commits → detect new SHA
T+150-180s │   ├── Fetch raw.githubusercontent.com/dashboard.html → swap DOM
T+150-180s │   ├── Indicator: [LIVE] → [SYNC] Syncing → [LIVE]
T+150-180s │   └── User sees commit's effect, no manual reload
```

Worst-case end-to-end: ~3 phút từ push đến browser visible. Best case (poll cycle align): ~30s.

6. Security — GitHub PAT

6.1 Tại sao cần PAT?

Repo maingocanh1702/MyMoneyWent **private** → GitHub API unauth request → **404** (deliberately hide existence of private repos). Polling JS không có credential → 404 mỗi lần poll → indicator stuck [ERR].

Fix: set GitHub PAT vào localStorage.github_pat. JS auth bằng Authorization: Bearer <token> header → API recognize anh có access → return 200 với commit SHA.

6.2 Classic vs Fine-grained tokens

Aspect	Classic	Fine-grained (Recommended)
Scope granularity	Coarse (repo = all repos)	Per-repo
Permissions	All-or-nothing	Per-permission (contents read-only OK)

Aspect	Classic	Fine-grained (Recommended)
Expiration	Optional	Required (max 1 year)
Audit	Limited	Better tracking
Blast radius if leaked	Toàn bộ user's repos	Chỉ repo được chọn

Em recommend Fine-grained: - Resource owner: maingocanh1702 - Repository access: **Only select** → MyMoneyWent - Permissions: - Contents: **Read-only** - Metadata: Read-only (auto) - Expiration: 30 days (rotate thường xuyên)

6.3 Setup procedure

1. <https://github.com/settings/tokens?type=beta> → Generate new token
2. Copy token (chỉ hiện 1 lần)
3. Browser DevTools console:

```
// Type "allow pasting" first if Chrome blocks paste
localStorage.setItem('github_pat', 'github_pat_XXXXXXX')
location.reload()
```

4. Verify: indicator [ERR] → [LIVE], console không còn 403/404

6.4 Token leakage handling

[WARNING] Nếu token đã expose (paste vào chat AI, screenshot console, push lên repo): 1. **Revoke ngay** ở github.com/settings/tokens → Delete 2. Audit recent repo activity ở Settings → Audit log 3. Tạo token mới với scope nhỏ hơn 4. Re-set localStorage với token mới

6.5 Storage location

Token chỉ lưu ở localStorage của browser: - **Same-origin:** Chỉ JS từ `mymoneywent-production.up.railway.app` đọc được - **Persistent across sessions:** Tồn tại sau khi tắt tab, đến khi anh manually clear - **Per-browser:** Không sync giữa browsers — set ở Chrome ≠ available ở Firefox - **KHÔNG send tới Railway:** Token chỉ dùng cho fetch outbound tới `api.github.com`, server-side không thấy

7. Troubleshooting

7.1 Indicator stuck [ERR] GitHub API 404

Nguyên nhân: Private repo + no auth.

Fix: Set fine-grained PAT theo §6.3.

7.2 Indicator chuyển [WARN] Rate limit

Nguyên nhân: - Unauth: 60 request/hour → polling 30s = 120/hr → hit limit sau 30 phút - Authed: 5000/hr → khó hit unless multi-tab + frequent force-refresh

Fix: - Set PAT để bump 60 → 5000/hr - Đợi ~1 giờ để rate limit window reset - Pause polling (Esc key) khi không cần realtime

7.3 Console “Failed to load resource: 403”

Nguyên nhân: Token revoked/expired/wrong scope.

Fix:

```
// Browser console
const t = localStorage.getItem('github_pat')
fetch('https://api.github.com/repos/maingocanh1702/MyMoneyWent', {
  headers: { Authorization: 'Bearer ' + t }
}).then(r => console.log('Token status:', r.status, r.statusText))
```

- 200 → token OK
- 401 → revoked, tạo mới
- 403 → rate limit hoặc scope thiếu, kiểm tra Contents=Read-only

7.4 Dashboard không refresh sau push

Check pipeline từng tầng:

```
# Tầng 1: GH Action có chạy không?
gh run list --workflow=dashboard.yml --limit=5

# Tầng 1: Bot's auto-rebuild commit có trên main?
git log --oneline -5 docs/dashboard.html

# Tầng 2: Railway có serve version mới?
curl -s https://mymoneywent-production.up.railway.app/dashboard | \
  grep -o "Updated [0-9-]* [0-9:]*"

# Tầng 3: Browser JS có lỗi không?
# DevTools Console → check 4xx/5xx errors
```

Nếu Tầng 1 fail → check .github/workflows/dashboard.yml syntax, repo Actions tab. Nếu Tầng 2 fail → check Railway dashboard, env vars, build logs. Nếu Tầng 3 fail → check PAT, browser cache, CORS errors.

7.5 Browser Chrome chặn paste vào Console

Lỗi: “Don’t paste code into the DevTools Console...”

Fix: Trong console, gõ tay (không paste) cụm allow pasting → Enter. Sau đó paste được trong session.

8. Commit history (timeline)

SHA	Date	Commit	Layer
9e561d3	2026-05-13	feat(dashboard): build from tracker with split local/remote git-state detection	Tầng 1
a5ea4c4	2026-05-13	ci(dashboard): auto-rebuild + sync workflows	Tầng 1
3f3cdf8	2026-05-13	chore(precommit): build-dashboard hook for instant local rebuild	Tầng 1
1edc7a5	2026-05-13	fix(dashboard): satisfy ruff + mypy strict on tests + scripts	Tầng 1

SHA	Date	Commit	Layer
10abff4	2026-05-13	feat/dashboard): Phase 3 – live in-browser poll-fetch update (PR #14)	Tầng 3
b6b711b	2026-05-13	feat/dashboard): serve /dashboard via FastAPI for Railway deploy	Tầng 2

Tổng ~1900 LOC across scripts/build-dashboard.py, .github/workflows/dashboard.yml, .pre-commit-config.yaml, main.py, và test files. 32 unit + 4 integration tests cover các thành phần chính.

9. Cải tiến tương lai

9.1 W0.10 v3-rich UI (stale, chưa merge)

Branch feat/dashboard-v3-rich có: - Chart.js mini chart (MVP trajectory đến Sept 2026) - Filter buttons (All/In-review/In-progress/Blocked/Not-started) + search input - Click PR row → mở GitHub branch - Click next-chip → search GitHub issues - Auto-refresh meta tag 60s (fallback nếu JS poll fail)

Blocker: Base branch trước F07 merge → cần cherry-pick 2 commits qua main, resolve overlap với W0.9 detect_git_state ở scripts/build-dashboard.py. 1-2 giờ work.

9.2 Custom domain tienvenoidau.com/dashboard

Phase 6 W6.2 sẽ setup DNS + SSL cho tienvenoidau.com → Railway. URL chuyển từ mymoneywent-production.up.railway.app/dashboard sang tienvenoidau.com/dashboard — match BRD.

9.3 Server-side push instead of client poll

Hiện tại 30s polling → có latency. Alternative: - **SSE (Server-Sent Events)**: Railway endpoint stream updates, browser receive instant - **WebSocket**: Bidirectional, dùng cho dashboard có interactive elements Trade-off: thêm complexity. Hiện tại 30s polling đủ tốt cho project-progress use case.

9.4 In-page rate limit display

Show số API calls remaining trong indicator: [LIVE] Live · 4923/5000 calls left. Giúp anh biết khi nào cần slow down.

9.5 Filter token-protected paths only

Khi localStorage.github_pat empty, fall back to polling raw URL (no auth needed cho public files) thay vì API. Trade-off: raw URL có CDN cache ~5min → realtime degrade.

10. Tổng kết

Dashboard realtime hoạt động qua **defense-in-depth 3 tầng**:

1. **Tầng 1 (server-side)** đảm bảo dashboard.html trên main luôn fresh — qua pre-commit hook (instant), GitHub Action (sau push), hourly cron (safety net)

2. **Tầng 2 (Railway)** serve `dashboard.html` qua `https` origin để JS polling có thể fetch GitHub API mà không bị CORS block
3. **Tầng 3 (browser)** poll GitHub API mỗi 30s, fetch raw HTML khi có commit mới, swap DOM preserve scroll

Mỗi tầng có **failure mode + fallback** rõ ràng → không có single point of failure.

Original goal đã shipped end-to-end 2026-05-13: anh mở 1 tab browser → để chạy cả ngày → tự thấy progress của F08/F02/... khi anh push code từ worktree Claude Code.

Changelog

Version	Date	Notes
v1.0.0	2026-05-13	Initial explainer doc — full 3-tier pipeline shipped via W0.9 + Phase 3 PR #14 + Railway serve commit b6b711b.